

Documento base del proyecto

App web de gestión de inventario multi-bodega

1. Objetivo del proyecto

Desarrollar una app web de gestión de inventario para una empresa que compra y vende productos, con capacidad de operar múltiples bodegas, controlar existencias en tiempo real, registrar compras y despachos, generar alertas operativas y ofrecer un dashboard con visibilidad financiera básica del negocio.

La aplicación debe ser utilizable en un entorno real de operación administrativa, con una arquitectura que permita crecer de forma ordenada, incorporar nuevas bodegas sin rediseños estructurales y mantener consistencia en el manejo del stock.

2. Alcance del sistema

La app cubrirá la operación administrativa principal relacionada con inventario y control económico básico del negocio. No se manejarán lotes, devoluciones ni procesos productivos en esta primera versión.

El sistema estará orientado a personal administrativo con acceso completo a la operación del sistema.

3. Problemas que el sistema debe resolver

1. Tener control confiable del stock por producto y por bodega.
 2. Registrar entradas y salidas de inventario de manera estructurada.
 3. Reducir errores manuales en movimientos de inventario.
 4. Generar alertas cuando un producto esté en nivel crítico o agotado.
 5. Dar visibilidad financiera básica sobre costos, valor del inventario, ingresos y margen estimado.
 6. Permitir que la empresa agregue nuevas bodegas sin cambios mayores en el sistema.
 7. Mostrar actualizaciones de stock y alertas en tiempo real dentro de la app.
-

4. Módulos del sistema

4.1 Productos

Permitirá crear, editar y consultar productos.

Campos base esperados:

- nombre
- código interno o SKU
- categoría
- costo
- precio de venta
- stock mínimo
- estado

4.2 Bodegas

Permitirá crear y administrar múltiples bodegas.

Campos base esperados:

- nombre
- ubicación o descripción
- estado

4.3 Inventario

Permitirá visualizar el stock actual por producto y por bodega, y consultar existencias consolidadas o separadas.

Debe incluir:

- stock por producto
- stock por bodega
- historial de movimientos
- ajustes manuales

4.4 Compras

Permitirá registrar el ingreso de inventario asociado a proveedores.

Debe incluir:

- proveedor

- fecha
- productos comprados
- cantidad por producto
- costo unitario
- bodega de entrada
- total de la compra

4.5 Proveedores

Permitirá crear, editar y consultar proveedores.

Campos base esperados:

- nombre
- identificación si aplica
- teléfono
- correo
- dirección
- estado

4.6 Despachos

Permitirá registrar la salida de productos por venta desde una sola bodega por operación.

Debe incluir:

- fecha
- bodega de salida
- productos despachados
- cantidad por producto
- precio de venta de referencia si aplica
- total estimado de la operación

4.7 Alertas

Permitirá generar y visualizar alertas operativas.

Alertas mínimas:

- stock bajo
- producto agotado
- inconsistencias básicas de inventario

4.8 Dashboard

Permitirá visualizar indicadores operativos y financieros básicos.

Debe mostrar al menos:

- valor total del inventario
- productos con stock bajo
- productos agotados
- compras recientes
- despachos recientes
- resumen por bodega
- productos más movidos

4.9 Finanzas básicas

Permitirá visualizar métricas simples pero útiles para administración.

Debe incluir al menos:

- dinero invertido en inventario
- costo acumulado de compras
- valor actual del inventario
- ingresos registrados o estimados por despachos
- margen bruto estimado
- margen promedio por producto o categoría

4.10 Usuarios

Todos los usuarios del sistema serán administrativos y tendrán acceso funcional completo en esta primera etapa.

5. Requisitos funcionales

1. El sistema debe permitir crear, editar y consultar productos.
2. El sistema debe permitir crear, editar y consultar bodegas.
3. El sistema debe permitir registrar stock por producto y por bodega.
4. El sistema debe permitir registrar compras y aumentar automáticamente el stock.
5. El sistema debe permitir registrar despachos y disminuir automáticamente el stock.
6. El sistema debe permitir ajustes manuales de inventario.
7. El sistema no debe permitir despachos que generen stock negativo.
8. El sistema debe permitir administrar proveedores.
9. El sistema debe almacenar y mostrar el historial de movimientos de inventario.
10. El sistema debe generar alertas por stock bajo o agotado.
11. El sistema debe mostrar cambios de stock y alertas en tiempo real.

12. El sistema debe mostrar indicadores financieros básicos del negocio.
 13. El sistema debe permitir agregar nuevas bodegas sin cambios estructurales del sistema.
-

6. Reglas de negocio

1. Un producto puede existir en múltiples bodegas.
 2. El stock se controla por la combinación producto + bodega.
 3. Una compra incrementa el stock del producto en la bodega seleccionada.
 4. Un despacho reduce el stock del producto en la bodega seleccionada.
 5. Cada despacho pertenece a una sola bodega.
 6. Un ajuste manual puede aumentar o disminuir stock, pero debe quedar registrado.
 7. El sistema no debe permitir stock negativo.
 8. Cuando el stock sea menor o igual al stock mínimo definido, se debe generar una alerta.
 9. Cuando el stock llegue a cero, se debe generar una alerta de agotado.
 10. El costo y el precio de venta del producto son editables manualmente por un administrador.
-

7. Requisitos no funcionales

1. La aplicación debe mantener consistencia del stock aun cuando existan operaciones concurrentes.
 2. La aplicación debe registrar trazabilidad suficiente de compras, despachos y ajustes.
 3. La aplicación debe responder adecuadamente en tiempo real para eventos de stock y alertas.
 4. La arquitectura debe ser modular y mantenible.
 5. La aplicación debe poder crecer en cantidad de productos, movimientos y bodegas sin requerir rediseño completo.
 6. La solución debe poder desplegarse en servidor o nube y operar de forma estable.
 7. La aplicación debe exponer observabilidad básica mediante logs y manejo de errores.
-

8. Arquitectura propuesta

La aplicación se implementará como un monolito modular. Esta decisión reduce complejidad inicial, facilita consistencia transaccional del inventario y acelera el desarrollo sin sacrificar capacidad de evolución.

8.1 Capas de arquitectura

- Capa API: endpoints HTTP y WebSocket.
- Capa de aplicación: casos de uso y servicios.
- Capa de dominio: reglas de negocio y entidades.
- Capa de infraestructura: base de datos, repositorios, configuración, logging y notificaciones.

8.2 Módulos internos sugeridos

- auth
 - users
 - products
 - warehouses
 - inventory
 - suppliers
 - purchases
 - dispatches
 - alerts
 - dashboard
 - finance
-

9. Stack tecnológico

9.1 Backend

- Rust
- Axum para API HTTP y WebSockets
- Tokio como runtime async
- Tower para middleware
- Serde para JSON
- SQLx para acceso a PostgreSQL
- PostgreSQL como base de datos principal
- tracing y tracing-subscriber para logging
- uuid para identificadores
- time para manejo de fechas y horas
- thiserror y anyhow para manejo de errores
- validator para validación de entrada si se requiere

Axum es una librería HTTP del ecosistema Tokio enfocada en ergonomía y modularidad, y Tokio provee la base async para red, timers, scheduling e I/O no bloqueante. SQLx es un toolkit async para Rust con soporte para PostgreSQL y validación de consultas en compilación. PostgreSQL es una base de datos relacional con transacciones e integridad adecuadas para un

sistema de inventario.

Fuentes: Tokio oficial, SQLx oficial, PostgreSQL oficial. (tokio.rs)

9.2 Frontend

Para que la aplicación sea completa y utilizable, se recomienda:

- React
- TypeScript
- Vite
- Tailwind CSS
- TanStack Query para consumo de API y sincronización de datos
- Zustand o Context API para estado local, según complejidad

9.3 Tiempo real

Se usarán WebSockets para:

- cambios de stock
- alertas en vivo
- actualización resumida del dashboard

Los WebSockets no serán la fuente de verdad. La fuente de verdad será la base de datos; WebSocket actuará como canal de notificación y refresco de interfaz.

9.4 Infraestructura y despliegue

- Docker para contenerización
- Docker Compose para entorno local y despliegue simple inicial
- Nginx como reverse proxy
- VPS o nube para despliegue inicial
- GitHub Actions o pipeline similar para CI/CD

10. Modelo de implementación

Fase 1

- definición detallada de entidades
- diseño de base de datos
- estructura base del backend en Rust

- autenticación básica
- módulo de productos
- módulo de bodegas
- módulo de proveedores

Fase 2

- módulo de inventario
- compras
- despachos
- movimientos de inventario
- reglas de stock y consistencia

Fase 3

- alertas
- WebSockets
- dashboard operativo
- dashboard financiero básico

Fase 4

- endurecimiento técnico
- pruebas
- observabilidad
- despliegue
- documentación operativa

11. Estrategia de desarrollo

El proyecto se desarrollará de manera iterativa.

En cada iteración se hará:

1. definición de requerimientos concretos del módulo
2. definición de reglas de negocio
3. diseño de datos
4. implementación backend
5. implementación frontend
6. pruebas
7. integración con módulos previos

Esta estrategia permite ajustar el sistema junto con los ingenieros industriales y traducir progresivamente necesidades del negocio a reglas de software.

12. Estrategia de despliegue

La aplicación podrá desplegarse en un entorno productivo siguiendo este flujo:

1. Base de datos PostgreSQL desplegada en servidor o servicio administrado.
 2. Backend Rust empaquetado en contenedor Docker.
 3. Frontend compilado y servido detrás de Nginx o mediante hosting estático con proxy al backend.
 4. Nginx configurado como reverse proxy para HTTP y WebSockets.
 5. Variables de entorno para conexión a base de datos, secretos y configuración.
 6. Migraciones ejecutadas antes de arrancar producción.
 7. Logs y monitoreo básico habilitados.
-

13. Resultado esperado

Al finalizar el proyecto, la empresa debe contar con una aplicación web utilizable que permita:

- administrar productos
 - administrar bodegas
 - registrar compras
 - registrar despachos
 - controlar stock por bodega
 - hacer ajustes manuales
 - recibir alertas
 - consultar dashboard operativo y financiero básico
 - operar el negocio con información más clara, consistente y centralizada
-

14. Decisión técnica consolidada

La solución se construirá como una app web full-stack con backend en Rust usando Axum, Tokio, PostgreSQL y SQLx, arquitectura monolítica modular, notificaciones en tiempo real mediante WebSockets y despliegue mediante contenedores para poder operar en un entorno real de inicio a fin.